

LLM Evaluation Methods

To build reliable LLM applications, we must create evaluation pipelines. An **LLM Evaluation (Eval) Method** is the specific mechanism we use to decide whether an LLM's output is good or not. It represents the actual procedure that takes an LLM's output and produces a judgment about it.

We build evaluation pipelines to determine whether our components, workflows, or applications are working correctly. However, the evaluation pipeline itself must be executed by a specific method. There are primarily three methods that can carry out and execute an evaluation pipeline:

1. **Programmatic / Deterministic Method:** The evaluation is executed entirely by a computer program (using code, such as Python).
2. **Human Method:** The evaluation is executed directly by a human who evaluates the outputs.
3. **Model-Graded / LLM-Graded Method:** The evaluation is executed by another LLM acting as the judge.

In our past session, we explored a practical use case where we acted as AI engineers for Zomato. We built an application where incoming emails were sent to an LLM to be categorized into one of three classes: **billing issues**, **technical issues**, or **general issues**. To evaluate how well this application worked, we used Python code to calculate its **accuracy**. Because a program calculated this metric, it is a prime example of a **Programmatic / Deterministic** evaluation method.

2. Programmatic / Deterministic Evaluation: Component-Level Retrieval Evaluation

To understand programmatic evaluation deeply, let us look at how we can evaluate a single component of a **Retrieval-Augmented Generation (RAG)** pipeline.

The Use Case Setup

Suppose we want to build a RAG chatbot for CampusX so that our users can interact with it and get immediate help, eliminating the need for us to manually reply to support emails. To ensure this chatbot is high quality, we want to evaluate it at the component level first, starting with the **Retriever**.

The fundamental role of a **Retriever** is simple: when it receives a question, it queries a vector database and retrieves the top (K) most relevant documents. We must verify if our Retriever is successfully fetching the correct documents.

Step 1: Define the Task and Target

- **Target:** A single component, which is the **Retriever**.
- **Task:** Evaluate whether our Retriever is working correctly.

Step 2: Define the Success Criteria

To evaluate the Retriever, we must quantify its performance. We do this using a critical retrieval metric: **Recall @ K**.

- **Definition of Recall @ K:** Out of all the correct, relevant items that exist in our database for a given query, how many did our system successfully retrieve within its top (K) results?

Illustrative Calculation of Recall @ K

Let us walk through an exact calculation:

1. **The User Question:** *"What are the prerequisites for ML course and how long it is?"*
2. **Vector Store State:** We have chunked all CampusX documents and stored them in our vector store.
3. **Ground Truth:** For this specific question, we know in advance that the correct answers reside in exactly two documents: **Document 1001** and **Document 1003**. These are the only correct documents that exist.
4. **Retriever Execution:** We send this question to our Retriever and configure it to retrieve (K = 5) documents.
5. **Retrieved Documents:** Our Retriever returns the following five documents: ``.
6. **Calculating the Recall:**
 - Total number of correct documents that exist in the database = (2) (1001 and 1003).
 - Number of correct documents retrieved by the system = (1) (1001).
 - $$\text{Recall} = \frac{\text{Correct Retrieved}}{\text{Total Correct Existing}} = \frac{1}{2} = 50\%$$

Our target is to have a Recall of (100%). Recall scores are bounded: they can never exceed (100%) ((1.0)) and can never fall below (0%). While other metrics such as **Precision** and **Rank** are also useful for evaluation, we will keep our current pipeline simple by focusing solely on **Recall @ K** as our success criteria.

Step 3: Build the Golden Data Set

To run this evaluation, we must construct a test dataset:

- We sample **50 to 100 questions** that we anticipate users will ask once the chatbot goes live on our website.
- We ensure high coverage by deliberately including normal questions, difficult questions, edge cases, and random questions to create a representative sample.
- We then bring in a **Human Expert** to manually search our database and locate the exact documents containing the correct answers for each question. This maps each question to its true correct documents, forming our ground-truth **Golden Data Set**.

Step 4: Define and Run the Evaluation Method

We write an automated Python program to execute this evaluation:

1. Our program takes all 50 questions from our Golden Data Set and sends them sequentially *only* to the Retriever (not the full chatbot pipeline) with ($K = 5$).
2. The Retriever fetches 5 documents for each question.
3. On a per-row (per-question) basis, our program compares the retrieved documents against the ground-truth document IDs in our Golden Data Set to compute the individual Recall.
 - For **Question 1**: The correct document is 1001. The Retriever successfully returns 1001 in its top 5. The Recall is (100%) ((1.0)).
 - For **Question 2**: The correct documents are 1001 and 1003. The Retriever returns 1001 but fails to return 1003. The Recall is (50%) ((0.5)).
4. The program repeats this calculation for all questions and computes the final average score.
5. Let's assume our final program output yields an average **Recall @ K** of **(67%)**.

Step 5: Analyze and Improve

With a (67%) success rate, we must analyze the specific questions where our Recall was poor. To improve our Retriever's performance, we can implement several strategies:

- **Upgrade the Embedding Model**: The current embedding model may be failing to capture the semantic meanings of queries accurately.
- **Query Expansion**: Instead of passing the raw user query directly to the Retriever, we can first pass it through an LLM to expand the query, and then send the expanded version to the Retriever.
- **Increase the value of (K)**: We can test increasing (K) from (5) to (10).
- **Implement Re-ranking**: We can retrieve a larger number of documents (e.g., top 10) and use a Re-ranker to bubble the most relevant ones to the top. If a correct document was originally at rank 8, a re-ranker can pull it up to rank 3, placing it within our top 5.

This entire execution is programmatic. No human is required to run the evaluation or calculate the scores, making this approach highly cost-effective since we avoid paying salaries for manual grading.

Note on Relevancy: Relevancy is multi-faceted. In this pipeline, we only measured one aspect (how many of the actual correct documents were fetched). Other aspects include measuring how many retrieved documents were completely useless (Precision) and analyzing if the retrieved documents were sorted in the correct rank order.

3. Human Evaluation Method: Evaluating End-to-End Application Quality

Some aspects of LLM applications are too nuanced for a computer program to evaluate. In these scenarios, we must design an evaluation pipeline executed by humans.

The Use Case Setup

We have a general RAG chatbot on the CampusX website that answers questions regarding upcoming course launches, fees, certificates, validity, and refunds. We want to evaluate the overall quality of this application.

Step 1: Define the Task and Target

- **Target:** The entire, end-to-end application (not a single component or isolated workflow).
- **Task:** Evaluate the **Helpfulness** of the generated answers.

We define **Helpfulness** as a composite of three characteristics: the answer must be **accurate**, written in the **correct tone**, and **complete** in itself.

Step 2: Define the Success Criteria

Because "helpfulness" is subjective and varies from business to business, there is no single out-of-the-box metric. We must define a custom evaluation **Rubric** on a scale from 1 to 5:

- **Grade 5 (Highly Helpful):** The answer is fully accurate, complete, and delivered in the correct tone.
- **Grade 3 (Partially Helpful):** The answer is only partially accurate or incomplete.
- **Grade 1 (Not Helpful):** The answer is entirely unhelpful, incorrect, or completely irrelevant.

Step 3: Build the Data Set

We compile a representative set of 50 sample questions covering various user scenarios (e.g., *"How long is the ML course?"*, *"Is the course right for me if I already know Python?"*, *"Can I pay the fee in installments?"*).

Unlike our programmatic evaluation dataset, this dataset contains **only one column: the questions**. It does not contain pre-defined correct answers because we are evaluating the live helpfulness of the chatbot's generation.

Step 4: Define and Run the Evaluation Method

Because a program cannot judge the nuance of helpfulness, our chosen evaluation method is **Human**.

1. We trigger the evaluation pipeline by sending each question from our dataset to the chatbot to generate an answer.
2. We seat a human grader and provide them with our rubric instructions.

3. For each test case, the human reviews the Question and the chatbot's generated Answer, and then assigns a score from 1 to 5 based on the rubric.
4. We repeat this across all questions and average the grades to obtain our final Helpfulness Score.

The Value of Multiple Graders

In professional setups, we often employ multiple human graders (e.g., **Grader A** and **Grader B**):

- If our graders consistently assign highly divergent scores to the same outputs (e.g., Grader A gives a score of 2 while Grader B gives a 4), this tells us there is **ambiguity in our rubric**.
- High agreement among graders indicates that our rubric instructions are exceptionally clear. High disagreement signals that the rubric is ambiguous and must be refined.

The Five Types of Human Evaluations in LLM Systems

Humans do not just evaluate systems by grading answers. In the LLM space, humans carry out five distinct types of evaluations:

1. **Direct Grading and Rating:** Humans evaluate a generated output against a predefined rubric and assign a score (as detailed in the chatbot helpfulness setup above).
2. **Red Teaming:** A dedicated group of humans intentionally attacks our LLM-based system to find security flaws, biases, or logical bypasses. When they succeed in breaking the system, they collect those inputs and send them to the development team to be patched before the model is launched.
3. **A/B Testing:** We deploy two different versions of our application in production. We allow real users (acting as our human evaluators) to interact with them and rate their experiences. The version that wins higher ratings is scaled up and deployed regionally.
4. **Golden Data Set & Rubric Creation:** Humans act as evaluators during the preparation phase by manually defining the criteria of what makes an answer "correct" and compiling the ground-truth test cases.
5. **Human in the Loop (HITL):** When a system handles highly complex cases where programmatic checks or LLM judges fall into a gray area or below a confidence threshold, the final judgment is routed to a human expert to make the final decision.

Trade-offs of Human Evaluations

- **Advantage:** High **reliability** and **trust**. A human brain is unmatched in its ability to understand context, linguistic nuances, and subtle errors.
 - **Disadvantage:** **High cost** and **poor scalability**. Scaling human evaluation to millions of production queries is financially and logistically impossible.
-

4. Model-Graded Evaluation: LLM-as-a-Judge

When programmatic evaluation is too rigid to handle subjective content, but human evaluation is too expensive to scale, we turn to the middle path: **Model-Graded Evaluation**. This technique is widely known as **LLM-as-a-Judge**, where we use an LLM as our evaluator.

The Case Study Setup

Let us set up a production scenario. We run an online platform called **CampusX UPSC** that prepares students for India's UPSC exam (one of the hardest exams in the world). The exam has three stages:

1. **Prelims**: Multiple-choice questions (MCQs).
2. **Mains**: Subjective, long-form essay questions.
3. **Interview**.

Evaluating Prelims mock tests is easy because we can grade MCQs programmatically. However, evaluating Mains mock tests is extremely difficult because they are subjective. If 10,000 students take our Mains mock test, we would have to hire hundreds of subject matter experts and pay them per-paper grading fees, which would wipe out our profit margins.

To solve this, we want to build an automated grading platform where an LLM evaluates these subjective essays using our grading rubrics at a fraction of the cost. However, before launching this platform to the public, we must prove that our LLM evaluator grades student papers as accurately as human experts do.

Step 1: Define the Task and Target

- **Target**: Our automated UPSC subjective grading application.
- **Task**: Evaluate whether our LLM grades subjective student answers exactly like human experts do.

Step 2: Define the Success Criteria

Our system is successful if its grading closely aligns with human grading. If the LLM awards scores that match human expert evaluations, we can confidently deploy the system.

Step 3: Define Rubrics and Build the Golden Data Set

Suppose our mock test contains three questions, such as:

- *Q1: "Ethical governance is impossible without administrative accountability. Discuss." (15 Marks)*

We bring in a human UPSC expert and ask them to define a grading **Rubric** listing the key dimensions an answer must address. For Q1, our expert defines five dimensions:

1. Discusses ethical governance and administrative accountability.

2. Explains the link between them.
3. Provides specific implementation mechanisms.
4. Cites relevant real-world examples.
5. Concludes with a balanced argument.

We then collect a sample of **50 to 100 student answers** to these questions. Our human expert reads each answer and, using the 5-point rubric, grades them and awards a final score (e.g., Student A gets 13/15; Student B gets 4/15). This mapped dataset serves as our ground-truth **Golden Data Set**.

Step 4: Define the Evaluation Method and Prompt

Our evaluation method is **LLM-Graded**. We bring in an LLM and act as the prompt engineer to design its grading behavior. We feed the LLM a structured prompt:

```
You are grading a UPSC mains answer against an evaluation rubric.

[Question Details]
Question: {Insert Question Text}
Total Marks: {Insert Max Marks}

[Evaluation Rubric]
{Insert 5-Dimension Rubric}

[Aspirant Answer]
{Insert Student's Submitted Answer}

Instructions:
For each dimension, decide whether the answer genuinely addresses it. Then allocate marks.
Do not reward verbosity, keyword stuffing, or confident assertions that lack substantiation.
Reward structure, relevant examples, and balanced argumentation.

Provide your output in the following format:
1. List of dimensions addressed.
2. Total marks awarded.
3. A one-sentence reasoning/justification for your score.
```

Step 5: Run, Compare, and Calculate the Metric

We pass our student answers through our LLM judge using the prompt above. This gives us two scores for every single student answer: the **Human Expert Score** and the **LLM Judge Score**.

Student Answer ID	Human Score	LLM Judge Score
Answer 1	13	12
Answer 2	4	8
Answer 3	8	8

To measure the alignment between the human and the LLM, we use the machine learning metric **Mean**

Absolute Error (MAE).

The MAE Formula

$$\text{MAE} = \frac{\sum_{i=1}^N |\text{Human Score}_i - \text{LLM Score}_i|}{N}$$

Where N is the total number of student answers (50 in our test run). Let's calculate this using our sample table:

- For Answer 1: $|13 - 12| = 1$
- For Answer 2: $|4 - 8| = 4$
- For Answer 3: $|8 - 8| = 0$
- Assuming we sum all absolute errors across our 50 test cases and the total sum is 115: $\text{MAE} = \frac{115}{50} = 2.3$

Interpreting the MAE

An MAE of **2.3** means that, on average, our LLM judge's score deviates from our human expert's score by (pm 2.3) marks.

Our ultimate engineering goal is to bring this MAE down as close to **0** as possible. An MAE of 0 indicates that our LLM evaluator grades papers with perfect alignment to our human experts.

To drive this error rate down, we can iterate on several system parameters:

- Implement a more powerful LLM model as the judge.
- Refine the system instructions and guidelines in our prompt.
- Work with our subject matter experts to make the grading rubric dimensions more explicit and less open to interpretation.

The Production Engineer Mindset

As production engineers, we must realize that building an LLM application is actually the easy part. The truly challenging and critical part is establishing robust evaluation systems to guarantee that our application works reliably every single time, in every scenario, before we ever deploy it to production.

5. Reference-Based vs. Reference-Free Evaluations

All evaluation pipelines can be categorized into one of two paradigms depending on the structure of their test data:

Reference-Based Evaluation

- **Definition:** An evaluation approach where we have a pre-defined, known "correct" answer (the reference) written down in advance for every single test case in our golden dataset. The system

evaluates quality by directly comparing the generated LLM output against this pre-recorded correct reference.

- **Examples from our session:**
 - **Our Retriever Evaluation:** We knew in advance the exact document IDs (1001, 1003) that held the correct answers. We evaluated the Retriever's output by comparing it directly to these pre-defined IDs.
 - **Our UPSC Grading System:** Our Golden Data Set contained the exact score assigned by the human expert for each specific student essay. We evaluated the LLM judge's output by comparing its scores directly to this known correct reference score.

Reference-Free Evaluation

- **Definition:** An evaluation approach where we do **not** have a pre-defined correct answer for each specific test case in our dataset. Instead, we evaluate the quality of the output directly on its own merits against a general, scaled standard or rubric.
- **Key Distinction:** In a reference-free setup, our rubric acts as a generalized quality standard rather than an item-specific correct answer key.
- **Example from our session:**
 - **Our Chatbot Helpfulness Evaluation:** Our evaluation dataset consisted purely of questions. There were no pre-recorded correct answers. The human evaluator read the chatbot's response and judged its quality on the fly, scoring it from 1 to 5 based entirely on their own judgment and our general rubric criteria.

Classification Rule

When analyzing any evaluation pipeline to determine if it is Reference-Based or Reference-Free, simply ask:

"Does my Golden Data Set contain a pre defined correct answer for each specific test item?"

- If **Yes** , It is a **Reference-Based Evaluation**.
 - If **No** , It is a **Reference-Free Evaluation**.
-

6. Offline vs. Online Evaluation

Every concept we have studied so far falls under the umbrella of **Offline Evaluation**. These are evaluations conducted in a controlled environment using static test datasets before our application goes live.

However, evaluation does not stop once our system is deployed. We must also evaluate our system's performance in production with real users. This is called **Online Evaluation**, and it will be the core

focus of our next session.